

PROXY PATTERN

Structural Design Pattern

Proxy Pattern

Problem: Sometimes you don't want to give direct access to an object because of security concerns, performance reasons, or the need to control how it's accessed.

Solution: The Proxy Pattern creates a stand-in (proxy) object that sits between the client and the real object. This proxy controls and manages all access to the real object.

Problem Statement

Imagine you're building an app that displays **high-resolution images** (like a photo gallery app). Loading a large image from disk takes time and memory. You don't want to load the image until the user actually needs to see it.

Without a proxy, the application would load every image immediately when the program starts – even images the user never views. This wastes:

- 🕒 Time (slow startup)
- 💾 Memory (loading unnecessary data)
- ⚡ Processing power (wasting resources)

With a proxy, the image is only loaded when the user actually tries to view it. The proxy acts as a placeholder until the real image is needed.

Proxy Pattern Benefits

Lazy Loading: The real object is created and loaded only when you actually need it, which saves memory and speeds up your application startup time.

Access Control: The proxy can check permissions and control who gets to access the real object – like a security guard checking IDs before letting people enter.

Adding Extra Features: Proxies can add useful features like logging (tracking who accessed what), caching (storing results for faster access), or validation all without changing the original object's code.

Keeping Things Organized: The real object focuses only on its main job (like displaying images), while the proxy handles all the extra stuff like loading, security checks, and caching. This keeps the code clean and organized.